

Competitive Programming

Section 18 Editorial: DSU And Minimum Spanning Trees

Competitive Programming Course

2026-07-05

Table of contents

1	Editorial overview	3
2	DSU reminder	4
3	A. Network Growth	5
3.1	Restatement	5
3.2	Constraints and traps	5
3.3	Observations	5
3.4	Algorithm	5
3.5	Correctness proof	6
3.6	Complexity	6
3.7	Full C++ solution	6
3.8	Full Python solution	7
4	B. Minimum Network	9
4.1	Restatement	9
4.2	Observations	9
4.3	Algorithm	9
4.4	Correctness proof	9
4.5	Complexity	10
4.6	Full C++ solution	10
4.7	Full Python solution	11
5	C. Cluster Split	13
5.1	Restatement	13
5.2	Observations	13
5.3	Algorithm	13
5.4	Correctness proof	13
5.5	Complexity	14
5.6	Full C++ solution	14
5.7	Full Python solution	15
6	Common wrong answers	17
7	Stress testing	18
8	What to carry forward	19

1 Editorial overview

This section has three DSU applications:

Problem	Main idea
A. Network Growth	maintain components under edge additions
B. Minimum Network	Kruskal MST
C. Cluster Split	stop Kruskal at k components

2 DSU reminder

DSU maintains connected components under additions.

Operations:

```
find(x): representative of x's component  
union(a,b): merge components if representatives differ
```

Invariants:

```
parent[root] == root  
size[root] is valid only for roots  
two vertices are connected iff find(a) == find(b)
```

Use path compression in `find` and union by size in `union`.

3 A. Network Growth

3.1 Restatement

Start with n isolated computers. After each cable is added, output the number of connected components and the size of the largest component.

3.2 Constraints and traps

The graph changes after every edge, so recomputing components with BFS after each update would be too slow.

Traps:

- repeated edges should not change the answer;
- self-loops may appear in the local tests and should not change the answer;
- component count decreases only after a successful merge.

3.3 Observations

DSU tells us whether the new edge joins two different components.

If it does:

```
components -= 1
new_size = merged component size
largest = max(largest, new_size)
```

If not, nothing changes.

3.4 Algorithm

```
initialize DSU with n singleton components
components = n
largest = 1

for each edge u,v:
    if find(u) != find(v):
        merged_size = union(u,v)
        components -= 1
        largest = max(largest, merged_size)
print components, largest
```

3.5 Correctness proof

Lemma 1. DSU components match graph connected components after every processed edge.

Proof. Initially both have n singleton components. When an edge connects two different graph components, graph connectivity merges exactly those two components, and DSU performs the same union. When an edge lies inside one component, neither graph connectivity nor DSU changes. $\square$

Lemma 2. `components` is the number of connected components after every edge.

Proof. By Lemma 1, DSU components are graph components. The counter starts at n and decreases exactly when two DSU components merge. $\square$

Lemma 3. `largest` is the size of the largest connected component after every edge.

Proof. The only way a component size changes is a successful union. The algorithm compares the merged size against the previous maximum. Unchanged edges cannot create a larger component. $\square$

Theorem. Every printed line is correct.

Proof. Follows from Lemmas 2 and 3. $\square$

3.6 Complexity

Time: almost $O(m)$
Memory: $O(n)$

3.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, sz;
    DSU(int n) : parent(n), sz(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }
    int unite(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return sz[ra];
```

```

        if (sz[ra] < sz[rb]) swap(ra, rb);
        parent[rb] = ra;
        sz[ra] += sz[rb];
        return sz[ra];
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    DSU dsu(n);
    int components = n, largest = 1;
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        if (dsu.find(u) != dsu.find(v)) {
            largest = max(largest, dsu.unite(u, v));
            components--;
        }
        cout << components << ' ' << largest << '\n';
    }
    return 0;
}

```

3.8 Full Python solution

```

import sys

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        a = self.find(a)
        b = self.find(b)

```

```

        if a == b:
            return self.size[a]
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
        return self.size[a]

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    dsu = DSU(n)
    components = n
    largest = 1
    out = []
    for _ in range(m):
        u = next(it) - 1
        v = next(it) - 1
        if dsu.find(u) != dsu.find(v):
            largest = max(largest, dsu.union(u, v))
            components -= 1
        out.append(f"{components} {largest}")
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

4 B. Minimum Network

4.1 Restatement

Choose a minimum-cost set of edges connecting all vertices. If the graph is disconnected, print IMPOSSIBLE.

4.2 Observations

This is a minimum spanning tree problem. Kruskal sorts edges by weight and takes an edge exactly when it connects two different DSU components.

Negative edge weights are fine. They simply appear early in sorted order and should be taken if they do not create a cycle.

4.3 Algorithm

```
sort edges by weight
total = 0
used = 0
for edge in sorted order:
    if union(u,v):
        total += weight
        used += 1
if used != n-1:
    print IMPOSSIBLE
else:
    print total
```

4.4 Correctness proof

Lemma 1. Kruskal never chooses an edge that creates a cycle.

Proof. DSU rejects an edge exactly when its endpoints are already in the same chosen-edge component. Adding such an edge would create a cycle. $\square$

Lemma 2. Every chosen Kruskal edge is safe for some MST.

Proof. When Kruskal chooses edge e , its endpoints are in different current components. Consider the cut separating one of those components from the rest. Since edges are processed in increasing weight, no lighter crossing edge remains unprocessed and usable. By the cut property, e is safe. $\square$

Theorem. If the algorithm chooses $n-1$ edges, their total cost is the MST cost; otherwise no spanning tree exists.

Proof. By Lemma 1, $n-1$ chosen edges form a tree connecting all vertices. By Lemma 2, each choice can be extended to an MST, so the final tree is minimum. If fewer than $n-1$ edges are chosen, the graph cannot connect all vertices. $\square$

4.5 Complexity

Time: $O(m \log m)$

Memory: $O(n + m)$

4.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, sz;
    DSU(int n) : parent(n), sz(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }
    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        parent[b] = a;
        sz[a] += sz[b];
        return true;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<tuple<long long, int, int>> edges;
```

```

for (int i = 0; i < m; i++) {
    int u, v;
    long long w;
    cin >> u >> v >> w;
    edges.push_back({w, u - 1, v - 1});
}
sort(edges.begin(), edges.end());
DSU dsu(n);
long long total = 0;
int used = 0;
for (auto [w, u, v] : edges) {
    if (dsu.unite(u, v)) {
        total += w;
        used++;
    }
}
if (used != n - 1) cout << "IMPOSSIBLE\n";
else cout << total << '\n';
return 0;
}

```

4.7 Full Python solution

```

import sys

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        a = self.find(a)
        b = self.find(b)
        if a == b:
            return False
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
        return True

```

```

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    edges = []
    for _ in range(m):
        u = next(it) - 1
        v = next(it) - 1
        w = next(it)
        edges.append((w, u, v))
    edges.sort()
    dsu = DSU(n)
    total = 0
    used = 0
    for w, u, v in edges:
        if dsu.union(u, v):
            total += w
            used += 1
    print("IMPOSSIBLE" if used != n - 1 else total)

if __name__ == "__main__":
    main()

```

5 C. Cluster Split

5.1 Restatement

Choose minimum total edge cost so that the vertices are divided into exactly k connected groups.

5.2 Observations

This is Kruskal stopped early.

Start with n groups. Each successful union reduces the number of groups by one. To end with k groups, we need exactly $n-k$ successful unions.

The same cut-property reasoning applies: at every step, the cheapest edge joining two current groups is safe.

5.3 Algorithm

```
sort edges by weight
components = n
total = 0
for edge in sorted order:
    if components == k:
        break
    if union(u,v):
        total += weight
        components -= 1
if components == k:
    print total
else:
    print IMPOSSIBLE
```

5.4 Correctness proof

Lemma 1. The algorithm maintains a forest with exactly `components` connected groups.

Proof. Initially there are n isolated vertices. A successful union adds an edge between two different groups, so it merges exactly two groups and creates no cycle. Rejected edges change nothing. $\square$

Lemma 2. Each chosen edge is safe for an optimal forest with the target number of groups.

Proof. At the moment an edge is chosen, it is the lightest available edge crossing between two current DSU components. By the same cut-property exchange used in Kruskal, replacing any heavier edge that connects these parts cannot increase the cost. Thus the choice is safe. $\square$

Theorem. If the algorithm reaches k components, the printed cost is the minimum possible cost for exactly k groups; otherwise it is impossible.

Proof. By Lemma 1, reaching k components gives a valid k -group forest. By Lemma 2, the greedy choices are all safe, so the forest is minimum. If all edges are exhausted before reaching k , no remaining edge can merge the remaining components, so no valid solution exists. $\square$

5.5 Complexity

Time: $O(m \log m)$
Memory: $O(n + m)$

5.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, sz;
    DSU(int n) : parent(n), sz(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }
    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        parent[b] = a;
        sz[a] += sz[b];
        return true;
    }
};

int main() {
    ios::sync_with_stdio(false);
```

```

cin.tie(nullptr);

int n, m, k;
if (!(cin >> n >> m >> k)) return 0;
vector<tuple<long long, int, int>> edges;
for (int i = 0; i < m; i++) {
    int u, v;
    long long w;
    cin >> u >> v >> w;
    edges.push_back({w, u - 1, v - 1});
}
sort(edges.begin(), edges.end());
DSU dsu(n);
int components = n;
long long total = 0;
for (auto [w, u, v] : edges) {
    if (components == k) break;
    if (dsu.unite(u, v)) {
        total += w;
        components--;
    }
}
if (components != k) cout << "IMPOSSIBLE\n";
else cout << total << '\n';
return 0;
}

```

5.7 Full Python solution

```

import sys

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        a = self.find(a)
        b = self.find(b)
        if a == b:
            return False

```

```

        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
        return True

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    k = next(it)
    edges = []
    for _ in range(m):
        u = next(it) - 1
        v = next(it) - 1
        w = next(it)
        edges.append((w, u, v))
    edges.sort()
    dsu = DSU(n)
    components = n
    total = 0
    for w, u, v in edges:
        if components == k:
            break
        if dsu.union(u, v):
            total += w
            components -= 1
    print("IMPOSSIBLE" if components != k else total)

if __name__ == "__main__":
    main()

```

6 Common wrong answers

- Updating component count on repeated or internal edges.
- Using BFS after every edge addition.
- Sorting MST edges in the wrong direction.
- Forgetting that MST costs may be negative.
- Stopping Cluster Split after choosing k edges instead of when k components remain.

7 Stress testing

For small graphs, DSU results can be checked against BFS after each update.

For MST problems, compare Kruskal against brute force over all subsets when $n \leq 8$, or against Prim's algorithm on connected small graphs.

For Cluster Split, brute force subsets can verify the best forest with exactly k components on tiny graphs.

8 What to carry forward

DSU is the standard tool for connectivity under edge additions. Kruskal is the standard first MST algorithm because DSU tells us whether an edge is safe to add.

The next graph sections move from connectivity to weighted shortest paths, where edge weights represent travel cost rather than construction cost.